SOUTH LAKE HOME EDUCATORS



Lesson 10

Core Mechanics IV: Play It Again

Semester 1, Week 10 - Game-Over + Restart Flow

At a Glance

Lesson 10 objectives & materials

By the end of today, students will:

- Refactor setup code into a reusable reset routine
- Introduce a flag that gates whether gameplay applies
- Implement a full, clean restart via a dedicated key
- Test a game that's genuinely replayable, not just winnable once

What We're Using Today

- Demo laptop with all 8 engines verified
- Godot, for the live lecture demo
- No new assets this week - logic and refactoring lesson
- Every engine involves a genuine refactor, not just additions
- Your own [YourName]Game project from Lesson 9 - not a new one

IGDA framework tie-in: Programming - state machines (introductory), code reuse/refactoring, input handling (intermediate).

Today's Agenda

0:00 - 0:05	Welcome back, Lesson 9 recap
0:05 - 0:20	The Last Missing Piece: What "Over" Should Actually Mean
0:20 - 0:40	One Setup, Two Doors In
0:40 - 0:55	The Freeze: A Flag That Gates Everything
0:55 - 1:00	Introduce lab exercise, hand out engine-specific handouts
LAB TIME	***— Play It Again —***
1:00 - 1:50	Build game-over freeze and restart flow
1:50 - 2:00	Exit check: full playthrough demos



Welcome Back

Lesson 9 recap - your game got a real scoreboard.

5 min

Welcome Back

5 min

Quick recap of Lesson 9:

- Score and lives, both visible on screen the whole time you play
- A real, persistent HUD - the promise from Lesson 6, finally kept
- Today: what "Game Over" actually means finally gets built

Open your own project from last week today, not a new one - and no new assets needed either, this is a logic and refactoring lesson.



The Last Missing Piece

What should "Game Over" actually mean?

15 min

"Over" Doesn't Mean Anything Yet

15 min

Right now, a message shows up when you win or lose - and nothing else changes. Enemies keep coming. You can keep shooting. Today, "over" finally means something real.

Two pieces, today: freezing the round, and a clear way to start a fresh one - both built from the exact same trick.

Discussion: think of any game you've played - what usually happens the instant you lose? (Enemies stop, controls lock, a menu or prompt appears, some kind of restart option shows up.)



One Setup, Two Doors In

The same setup code, callable a second way.

20 min

Give Setup a Second Doorway

20 min

Your game already has code that sets everything up - running once since Lesson 5. Today, that same code gets a second way to run: on demand, whenever R is pressed.

Godot demo: GameManager gets start_game() (reset score, lives, enemies, then call spawn_enemies() on Main). GameManager's _ready() just calls it once, deferred (start_game.call_deferred()). Now start_game() can be called again, any time, from the restart key.

Nothing about the setup logic itself changed - it just got a second doorway in.

Discussion: why is one piece of setup code with two ways in better than writing it twice? (Change it once, not remember to update two copies.)



The Freeze

A single flag, checked in the right place.

15 min

One Flag Gates Everything

15 min

Restarting is only half of today. The other half: actually stopping the game once it's over, so nothing keeps happening behind the win/lose message.

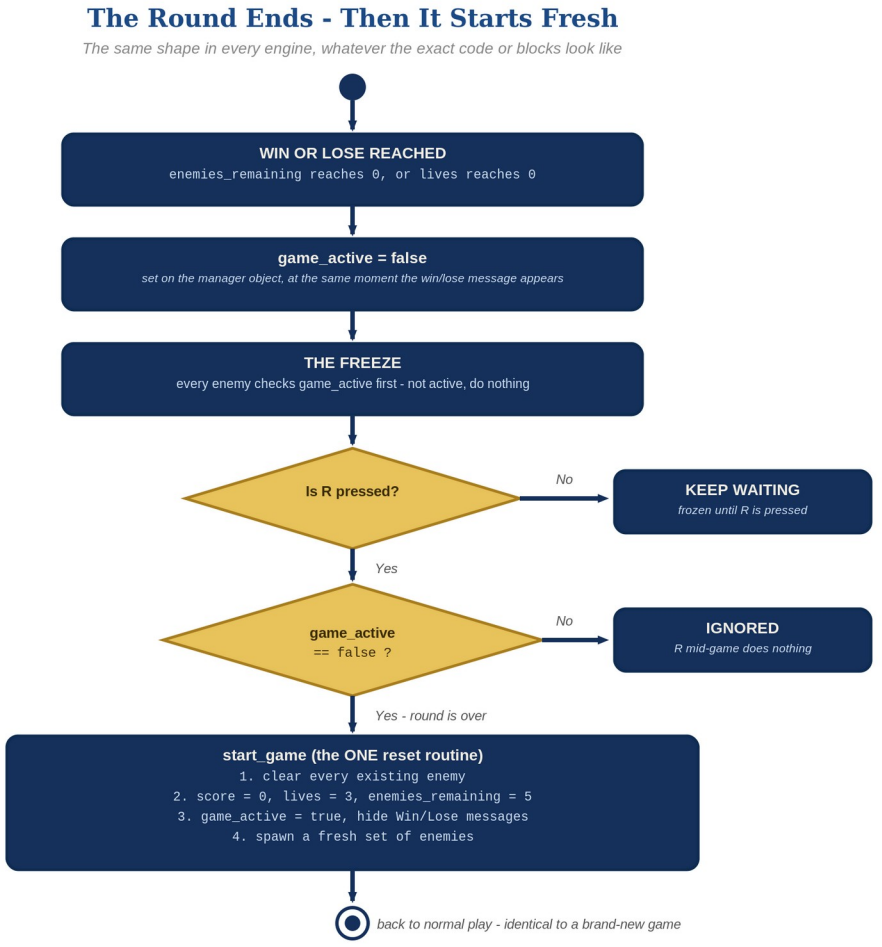


game_active (on GameManager) starts true, flips false at win/lose. At the very top of enemy movement code: if the game isn't active, don't do anything else this frame.



Discussion: what would happen if we forgot to check game_active somewhere - say, in the bullet-hits-enemy code? (That specific thing would keep happening even after game-over - test for this today.)

The Whole Round, One Picture



Freeze, ask, reset

- The round ends: game_active becomes false and everything freezes.
- R pressed? Round over? Only a double yes runs start_game().
- R mid-game is ignored - the second check blocks it.
- start_game() is the same routine that runs at launch: one setup, two doorways.

What Is Game State?

Game state is everything your game remembers right now.

Numbers and flags

score, lives, enemies_remaining, game_active

Where things are

player position, enemy positions, what is visible or hidden

How things look

costume or sprite, color, size, animations mid-play

What is playing

sounds, timers, tweens, messages on screen

Until today, your game started once and ended once. A restart means ALL of it has to go back to how it was at the start.

The Restart Audit

Two lists. Everything must be on one of them, and reset by start_game.

1. Everything that happens at START

- Scratch: EVERY "when green flag clicked" - all sprites and the stage
- Godot: every _ready()
- GameMaker: every Create event
- Unity: every Awake() and Start()

2. Everything that happens at the END

- Win and lose messages
- Animations, color or costume changes, sounds
- Anything hidden, moved, or stopped
- Anything extra YOU added - the handout can't know about it

If it changed when the round ended, something must change it back when the round restarts.



Today's Exercise & Lab

Play It Again - freeze and restart flow.

5 min

Play It Again

5 min

When the round ends, the game freezes. Pressing R resets everything and starts a new round - built on top of your existing project.

Level 1 - Minimal

Round ends -> game freezes; R resets and starts a new round.

MVP - a complete finish for today.

Level 2 - Complete

A restart is a genuinely clean slate, any number of times in a row.

Everything works end to end.

Level 3 - Refined

A transition effect, tighter reset structure, and a real bug to find and fix.

Ready to share.



Test in both directions and both end-states: win then restart, lose then restart, and confirm R does NOTHING while a round is still active.

Lab Time Notes

- Most important check: a full round-trip, twice - win then restart, and separately lose then restart. Both should leave an identical, clean starting state.
- Most likely mistake: old enemies not cleared before respawning on restart, leading to doubled enemies after the first restart.
- Second most likely mistake: R doing something mid-game because the game_active check was left out of the restart-key logic - test this directly.
- Game state drift: anything students added beyond the handout (extra green-flag scripts, animations, color changes, hidden objects) is not reset by the handout steps - use the Restart Audit checklist.
- Level 3 bug hunt: a round with some kills and some escapes ends with no win and no loss, and R does nothing. Early finishers should find it and fix it their own way.
- This is a satisfying lesson to finish - encourage a full playthrough (win, restart, lose, restart) as part of the share-out.



Genuinely Replayable!

A clean freeze and a real restart, built almost entirely by reusing code you already trust.

Next week: multiple waves - the Level 2 Complete milestone.

Try a Level 3 idea at home if you'd like - a transition effect is a fun, visible one.